

# Schéma d'architecture technique — solution MCP (analyse vidéo)

Document compagnon de `note-benefices-limites.md` (§5) et de `etude-faisabilite-couts.md`. Le schéma mermaid ci-dessous est la source ; il est rendu en image dans les slides.

## 1. Rappel MCP en 5 lignes (pour la note et pour le jury)

Le **Model Context Protocol** (standard ouvert initié par Anthropic fin 2024, largement adopté depuis dans l'écosystème des agents — les principaux fournisseurs de modèles et d'outils ont annoncé leur support courant 2025) normalise la façon dont un agent (« hôte ») découvre et appelle des capacités externes via des **serveurs MCP** exposant trois primitives : **tools** (actions), **resources** (données lisibles), **prompts** (gabarits). Transports : stdio en local, HTTP streamable en distant. Implémentation Python de référence pour nous : **FastMCP**.

**Argument d'architecture** : chaque serveur MCP est réutilisable par n'importe quel agent futur de la FFE (pas seulement le nôtre), indépendamment du framework d'orchestration — c'est la modularité demandée.

## 2. Le schéma

```
flowchart TB
    subgraph Hote["HÔTE MCP – Agent FFE (LangGraph, existant du POC)"]
        AG["Client MCP<br/>(découverte + appels d'outils)"]
    end

    subgraph Serveurs["Serveurs MCP (FastMCP, HTTP streamable)"]
        S1["srv-youtube<br/>tools: rechercher_videos,<br/>obtenir_transcript,<br/>obtenir_metadonnees"]
        S2["srv-vision<br/>tools: lancer_analyse(video_ref),<br/>etat_analyse(job_id), extraire_frames,<br/>detecter_echiquier,<br/>frame_vers_fen"]
        S3["srv-chess-tools<br/>tools: valider_fen, evaluer_position,<br/>coups_theoriques (réutilise le POC)"]
        S4["srv-connaissances<br/>tools: recherche_vectorielle, positions_video(fen)<br/>resources: fiches d'ouvertures"]
    end

    subgraph Pipeline["Pipeline d'ingestion BATCH (hors MCP – assumé)"]
        Q["File de jobs<br/>(Redis/queue)"]
        W1["Workers extraction frames<br/>(ffmpeg, 1 frame/5 s)"]
        W2["Workers vision<br/>(détection 2D OpenCV → FEN,<br/>option V2 : modèle entraîné)"]
    end
```

```

W3["Workers alignement<br/>(transcript ↔ positions ↔ timestamps)"]
end

subgraph Stockage["Stockage"]
  OS["Objet (S3/minio)<br/>frames clés uniquement"]
  MG["MongoDB<br/>jobs, métadonnées vidéos,<br/>positions+timestamps"]
  MV["Milvus<br/>vecteurs (chunks transcript,<br/>positions décrites)"]
end

AG --> S1
AG --> S2
AG --> S3
AG --> S4
S2 -- "déclenche / interroge" --> Q
Q --> W1 --> W2 --> W3
W1 --> OS
W2 --> MG
W3 --> MG
W3 --> MV
S4 --> MV
S4 --> MG
S1 -. "API YouTube officielle<br/>(métadonnées, jamais les fichiers)" .->
YT["YouTube Data API v3"]

```

### 3. Composants (tableau à intégrer dans la note)

| Composant         | Rôle                                           | Techno pressentie                       | Point clé                                                                   |
|-------------------|------------------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------|
| Hôte/agent        | Orchestration, dialogue élève                  | LangGraph (existant POC)                | Devient client MCP : aucun couplage direct aux implémentations              |
| srv-youtube       | Découverte de contenus + transcripts           | FastMCP + API YouTube                   | Respect CGU : métadonnées/transcripts, filtre licence CC                    |
| srv-vision        | Pilotage des analyses (asynchrone)             | FastMCP                                 | <b>lancer_analyse retourne un job_id</b> : MCP pilote, le batch exécute     |
| srv-chess-tools   | Validation FEN, éval, théorie                  | FastMCP encapsulant les services du POC | Réutilisation directe — argument fort                                       |
| srv-connaissances | Recherche sémantique + jointure position↔vidéo | FastMCP + pymilvus/Mongo                | La clé pivot reste le FEN normalisé                                         |
| File + workers    | Ingestion de masse                             | Redis + workers Python (ffmpeg, OpenCV) | ≈ 3 min CPU/vidéo → <b>~20 vidéos/heure/worker</b> (étude §1)               |
| Stockage objet    | Frames clés uniquement (pas les vidéos)        | minio local / S3 prod                   | ≈ <b>3 Mo/vidéo</b> (≈ 30 frames clés × 100 Ko — jamais les fichiers vidéo) |

## 4. Décisions d'architecture à défendre

1. **MCP pour l'interface, batch pour la masse** : un agent n'a pas à attendre 3 min d'analyse ; il déclenche (`lancer_analyse`) et consulte (`etat_analyse`). Honnêteté architecturale = crédibilité.
2. **On ne stocke jamais les fichiers vidéo** (CGU) : frames clés + positions + timestamps + transcripts. Le « stocke les vidéos » de la demande initiale est requalifié en « stocke les *références et produits d'analyse* des vidéos » — à expliciter dans la note.
3. **Idempotence** : une vidéo = un hash ; ré-analyse uniquement si version du pipeline change.
4. **Sécurité** : serveurs MCP derrière auth (token de service), pas d'exécution arbitraire, quotas par outil ; données en UE.
5. **Évolutivité** : passer du régime 2D (OpenCV pur, CPU) au régime 3D (modèle vision, GPU) = remplacer un worker, sans toucher ni aux serveurs MCP ni à l'agent.

## 5. Flux type (à raconter sur le slide 14)

Élève sur la position Italienne → agent appelle `positions_video(fen)` (srv-connaissances) → « cette position est expliquée à 4 min 32 dans la vidéo X (licence CC) » → lien horodaté dans l'UI. Si position inconnue → l'agent peut proposer `lancer_analyse` sur les meilleures vidéos candidates (srv-youtube) → disponible quelques minutes plus tard.